



High-Level Synthesis for Edge-AI Applications

Name	Nader Yahia
Matriculation	2210026
Supervisor	Prof. Dr.rer.nat. Achim Rettberg

Introduction

What is Edge AI?

- Execution of artificial intelligence models directly on edge devices (e.g., sensors, embedded systems, FPGA/SoC), close to the data source [1].

Why Edge AI?

- Low latency (real-time response required).
- Reduced bandwidth usage (no continuous cloud communication).
- Lower system dependency on cloud infrastructure.
- Improved privacy (data stays local).

What is Key edge constraints ?

- Very low power budget.
- Limited memory (on-chip bottlenecks).
- Limited communication bandwidth.
- Predictable real-time response requirements.

FPGAs represent a well-suited platform due to their balance of performance, energy efficiency, and reconfigurability [2].

Motivation For HLS

- **RTL Design Complexity:** Manual RTL design is time-consuming and requires high expertise.
- **Low Design Productivity:** Long development cycles limit fast prototyping and iteration.
- **Need for Design Space Exploration:** Rapid evaluation of optimizations (PIPELINE, UNROLL, PARTITION).
- **Bridging Software–Hardware Gap:** HLS bridges C/C++ algorithm design with FPGA RTL implementation.
- **Faster Time-to-Deployment:** HLS accelerates prototyping and iteration for Edge AI applications.

Model-to-Deployment Pipeline

- **Stage 1:** AI Model trained and exported (ONNX, quantized).
- **Stage 2:** C/C++ fixed-point model with optimizations.
- **Stage 3:** HLS converts C/C++ $\rightarrow$ RTL with scheduling & synthesis.
- **Stage 4:** RTL implementation (place & route, timing closure).
- **Stage 5:** FPGA-based accelerator executes AI inference.

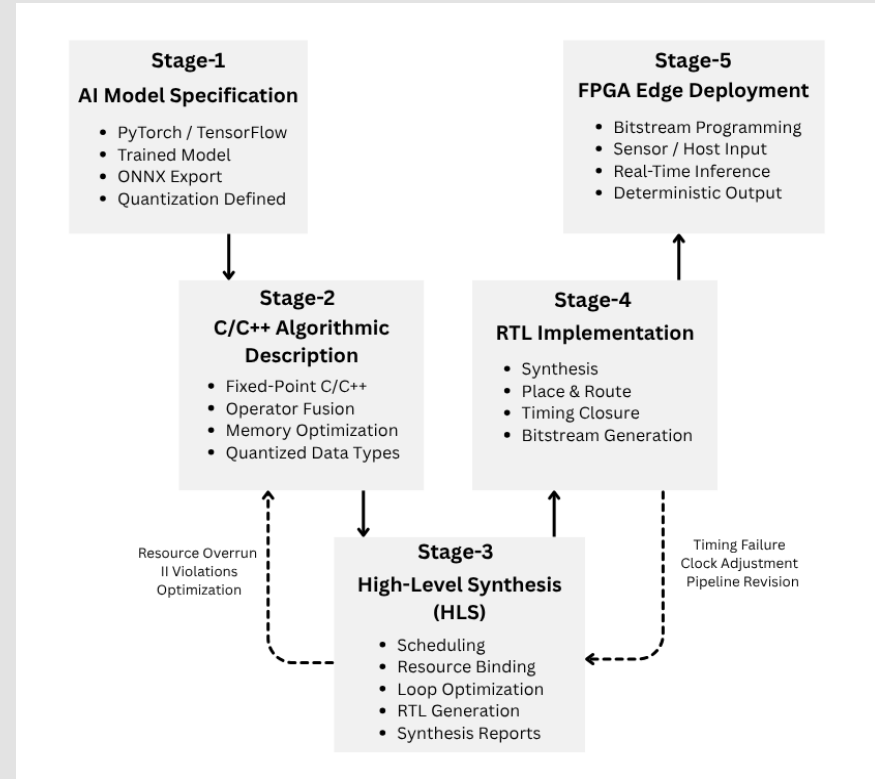


Fig.1 : Model-to-Deployment Pipeline

[1],[5]

HLS Compilation Flow

1. Code Parsing

- C/C++ → Abstract Syntax Tree (AST).

2. CFG/DFG Graph Construction

- CFG: describes execution control (loops, branches)
- DFG: describes computation dependencies

3. Scheduling

- DFG → Scheduled DFG.
- Assigns each operation to a clock cycle.

4. Resource Allocation + Binding (Hardware Mapping)

- Scheduled DFG → Hardware Mapping.
- Allocation: number of DSPs, ALUs, memory ports.
- Binding: Assign operations → specific hardware units.

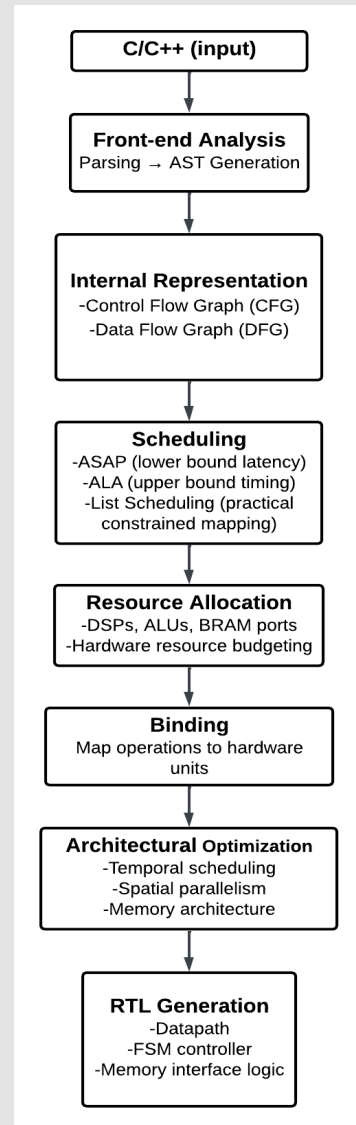


Fig.2: HLS Compilation Flow

HLS Compilation Flow

5. Loop Optimizations (Parallelism Extraction)

- II = initiation interval
- Pipelining: overlap loop iterations
- Unrolling: replicate loop body → spatial parallelism
- Dataflow: pipeline entire system into stages

Trade-offs

Latency ↔ Throughput ↔ Area (DSP/LUT/BRAM usage).

6. RTL Generation (Final Hardware)

- Mapped Graph → RTL (Verilog/VHDL)
- Datapath: DSPs, registers, multiplexers
- Control: FSM derived from schedule
- Memory: BRAM / DRAM interface logic

Behavioral model → fully synthesizable FPGA architecture [3]

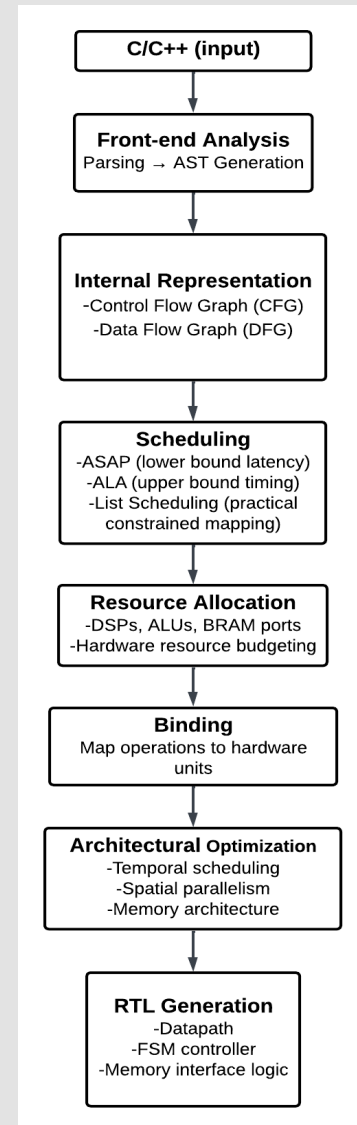


Fig.2: HLS Compilation Flow

Implementation Methodology

Objective

Accelerate a CNN 2D convolution kernel using HLS.

Optimization Flow



Development Environment	Evaluation Metrics
Artix-7 (xc7a100t-csg324-1)	Clock period
Vitis HLS 2022.2	Latency
C/C++ HLS	DSP utilization
100 MHz (10 ns constraint)	LUT utilization
Vivado IP Flow Target	FF utilization

Results

Performance Comparison

Metric	Baseline	PIPELINE	UNROLL
Clock (ns)	7.11	7.29	6.90
Latency (ms)	22.176	10.368	3.600
DSP	1	144	36
LUT	760	7705	6466
FF	260	3123	5691

Key observations

- PIPELINE reduced latency by approximately 53%.
- UNROLL reduced latency by approximately 84%.
- Both optimizations increased hardware resource utilization.
- UNROLL achieved the best performance-resource balance in this study.

Discussion

Baseline Model (Sequential)

- Each MAC is executed one after another.
- one shared multiplier and adder used.
- Highest latency and least resources utilized

```
sum += input[ic][i + ki][j + kj] * weight[oc][ic][ki][kj];
```

Pipeline Model

- Next MAC starts before previous finishes.
- Operations are overlapped in time.
- Latency: 22.17 ms $\rightarrow$ 10.36 ms ($\downarrow$ ~53%)
- DSP: 1 $\rightarrow$ 144 ($\uparrow$)

```
for (int ki = 0; ki < K; ki++) {  
    #pragma HLS PIPELINE II=1  
    sum += input[...] * weight[...];  
}
```

Unroll Model

- All MAC operations inside loop run in parallel.
- Multiple multipliers (DSPs) are instantiated.
- Latency: 22.17 ms $\rightarrow$ 3.60 ms ($\downarrow$ ~84%)
- DSP: 1 $\rightarrow$ 35 ($\uparrow$)

```
for (int ki = 0; ki < K; ki++) {  
    #pragma HLS UNROLL  
    sum += input[...] * weight[...];  
}
```

Conclusion

- Edge AI requires hardware accelerators that deliver high performance under strict power, latency, and resource constraints.
- High-Level Synthesis provides a practical design methodology fills the gap of RTL complexity, significantly accelerating the edge AI applications deployment.
- CNN convolution was used as a representative Edge AI workload to demonstrate how HLS directives influence hardware architecture and performance.
- These obtained results confirm that HLS support methodical hardware design exploration, thereby establishing it as a cornerstone of the Edge AI acceleration FPGA platforms.

[1]–[5]

Refereneces

- [1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016, doi: 10.1109/JIOT.2016.2579198.
- [2] E. Nurvitadhi, G. Venkatesh, J. Sim, and D. Marr, “Can FPGAs beat GPUs in accelerating next-generation deep neural networks?” in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2017, doi: 10.1145/3020078.3021736.
- [3] Advanced Micro Devices Inc. (AMD), *Vitis High-Level Synthesis User Guide*, 2023. [Online]. Available: <https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vitis/vitis-hls.html>.
- [4] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding,” *Proc. Int. Conf. Learning Representations (ICLR)*, 2016. [Online]. Available: <https://arxiv.org/abs/1510.00149>.
- [5] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, “Efficient processing of deep neural networks: A tutorial and survey,” *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017, doi: 10.1109/JPROC.2017.2761740.

Q & A